



Artic: AI-oriented Real-time Communication for MLLM Video Assistant

Jiangkai Wu, Zhiyuan Ren, Junquan Zhong, Liming Liu, Xingong Zhang



汇报人：陆俊安
2026.09.28

作者

研究方向:

生成式 AI 与视频传输的结合

导师: 张行功, 北大王选计算机研究所副教授

近期工作:

- ZGaming (SIGCOMM 2023)
用画面预测缩短云游戏交互等待
- Promptus (AAAI 2026)
探索用可学习提示词传输视频
- Chat with AI (HotNets 2025)
讨论 AI 视频助手的实时通信问题



Jiangkai Wu
(吴将凯)

<https://jiangkaiwu.github.io/>

目录

01 研究背景与挑战

02 方法与系统设计

03 实验结果

04 思考

研究背景：视频如何在网络上传输



码率：发送端每秒要发送的数据量（可控） 带宽：网络每秒能承受的数据量（不可控）

码率高于带宽 → 网络排队 → 出现延迟和卡顿

码率压缩得太低 → 视频质量下降，比如模糊和细节丢失

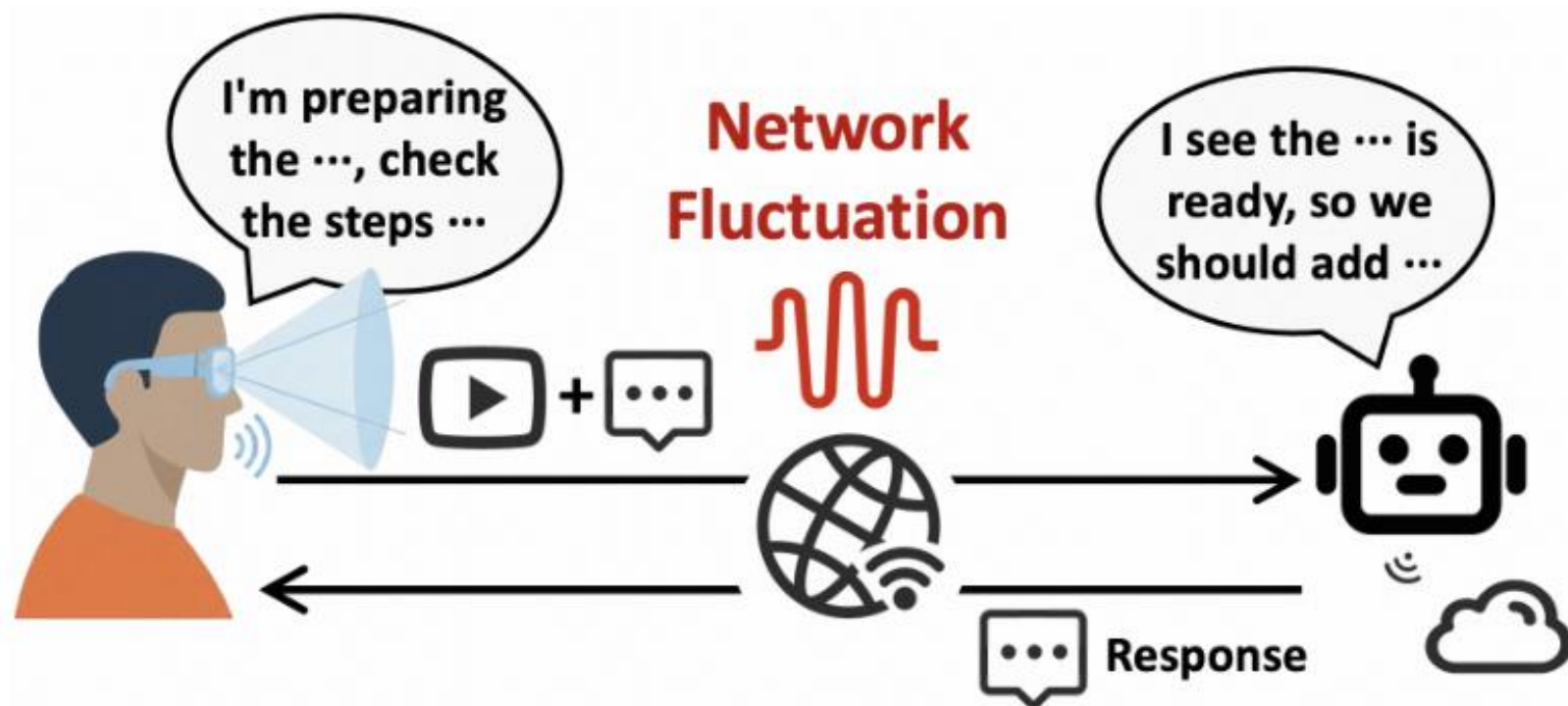
研究背景：AI视频助手的使用场景

AI 视频助手是一种新的实时通信范式。用户将视频和音频持续发送给多模态大模型进行思考，大模型再向用户反馈音频。

传统的实时通信场景：在线会议和视频通话。

视频助手的特征：

- 接收端从人变为机器
- 轻量客户端，云端计算
- 长时间工作，主动响应
- 行为随机，**带宽波动剧烈**



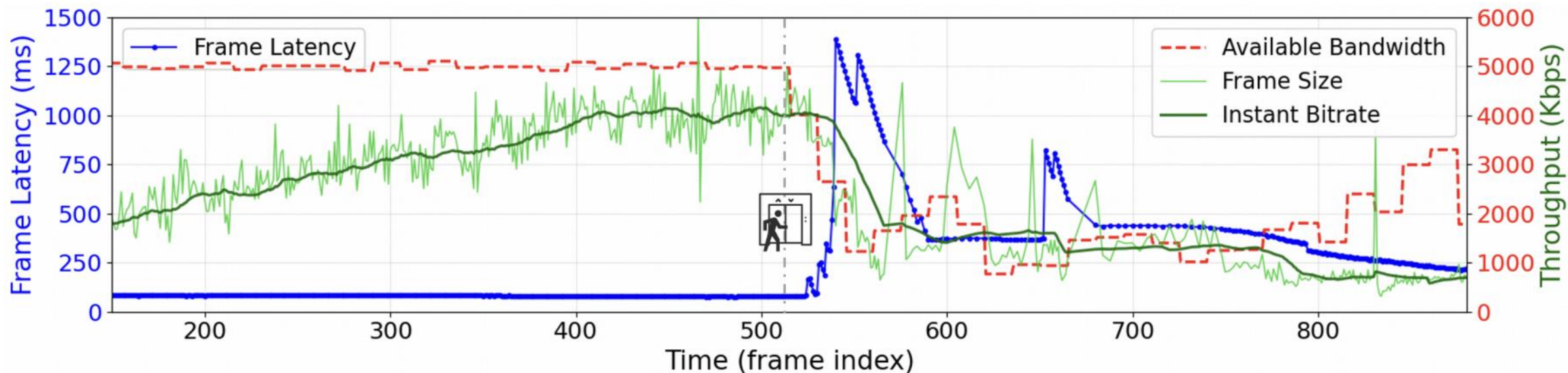
研究背景：新场景和旧框架之间的矛盾

场景	传统实时通信	AI视频助手
服务质量重点	保真度（清晰度）	模型响应准确率
	流畅性	
	一致性	
	时延（容忍时延较长）	时延（容忍时间更短）

核心矛盾：服务质量关注的重点从人类感知质量转变为模型响应速度和准确率。而为旧框架在新场景的表现不佳。于是，希望构建一个专门服务AI视频助手的新框架。

挑战1：移动性和随机用户行为造成带宽波动

真实 5G 上行轨迹：进入电梯时，可用带宽从约 5 Mbps 跌到 1.23 Mbps。



拥塞控制约 1.17 s 后才调整；视频帧时延最高到 1389 ms。

传统拥塞控制会在带宽足够时尽可能的提升码率，这导致在带宽突变时无法快速调整。

从而使端到端时延增加

挑战2：低码率让模型失去关键信息

恶劣网络环境下，码率不得不降低。

商品文字被压糊后，同一问题得到错误答案；换更强模型仍然答错，说明关键信息已经在压缩中丢失，提升模型能力没有作用。



Query: Please help me check the products I am restocking.



Response:

(a) 3254 Kbps, MLLM Live



The product you are restocking is vanilla Greek yogurt. ✓

(b) 968 Kbps, MLLM Live



The product you are restocking is Grekisk Yoghurt. ✓

(c) 429 Kbps, MLLM Live



The product in front of you is fresh cheese. ✗

(d) 429 Kbps, MLLM Pro



Okay, you are now picking up the mashed potatoes. ✗

额外挑战：面向人类感知需求构建的数据集无法评估视频退化如何影响模型准确率，对视频退化敏感的问题仅有8%

观察：从两种失效里找到可用的现象

观察到：模型的回答在一定码率后趋于稳定，重要信息又常集中在局部区域。

(a) 3254 Kbps, MLLM Live



The product you are restocking
is vanilla Greek yogurt. ✓

(b) 968 Kbps, MLLM Live



The product you are restocking
is Grekisk Yoghurt. ✓

① 答案已经正确时，
继续提升带宽占用没有收益。

② 带宽有限时，
把更多带宽分配给重要区域。

对于数据偏向的问题，则只能尝试构建退化敏感的数据集

设计：从实验发现到本文设计

实验发现

本文设计

回答在足够码率上趋于稳定



ReCapABR：置信度控制总码率

低码率损伤常集中在局部



ZeCoStream：按对话给局部更多质量

普通问答题难测出画质退化



DeViBench：退化敏感的问答基准

设计：ReCapABR 调整码率

让大模型自行判断码率是否足够，并返回置信度（0到1）

置信度 < 0.8 且带宽充足

→ 提高码率，避免细节丢失

模型已有把握（置信度 > 0.8）

→ 停止提升码率，留出余量应对波动

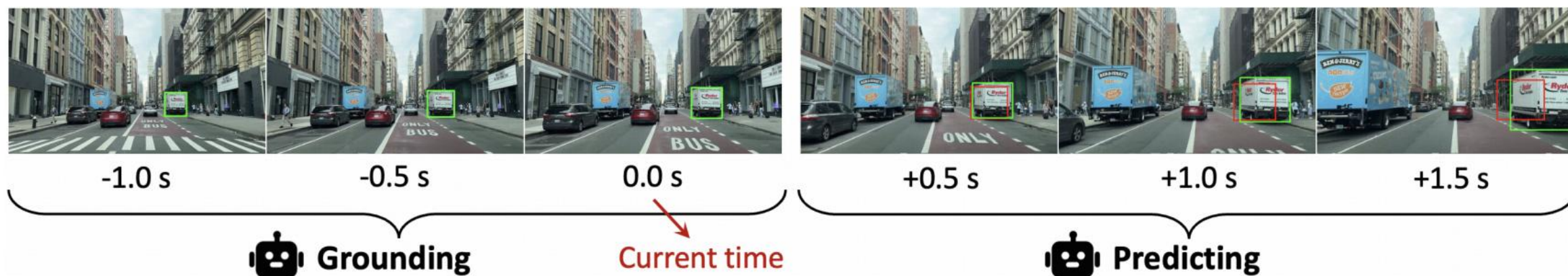
带宽低于当前发送码率

→ 降码率保延迟，信息损失问题交由ZeCoStream处理

如果带宽实在很低，则视频交给ZeCoStream处理，只传输重要区域。

设计：ZeCoStream 预测重点

大模型根据对话找到目标位置，再预测它未来 1.5 秒的位置。



(a) Baseline: 414 Kbps (b) Ours: 404 Kbps (c) QP from MLLM feedback



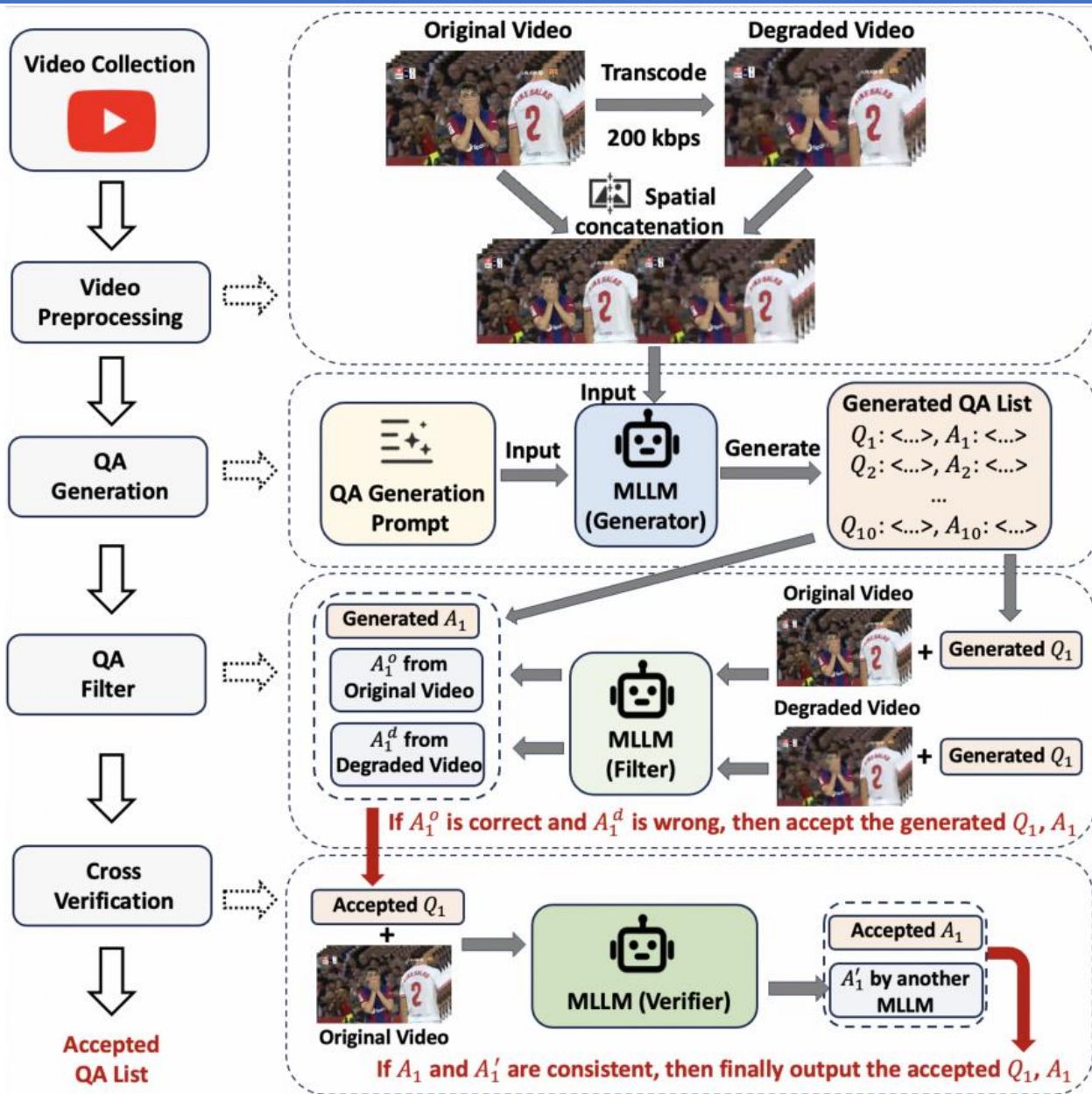
“What is the license plate number of the black SUV in the center?”

Baseline: I71604Y ✗ Ours: T716049C ✓

从预测块中心向周围，量化程度逐渐提升，以达到保留重要信息的目的。

量化可能导致损失一些现在不重要但是之后会使用的信息，所以该模块只在带宽不足且模型需要细节时启用。

设计：DeViBench 构造符合场景的数据集



从已有数据集中得到高清视频，将其压缩得到高清版和约200 Kbps码率的低清版本。

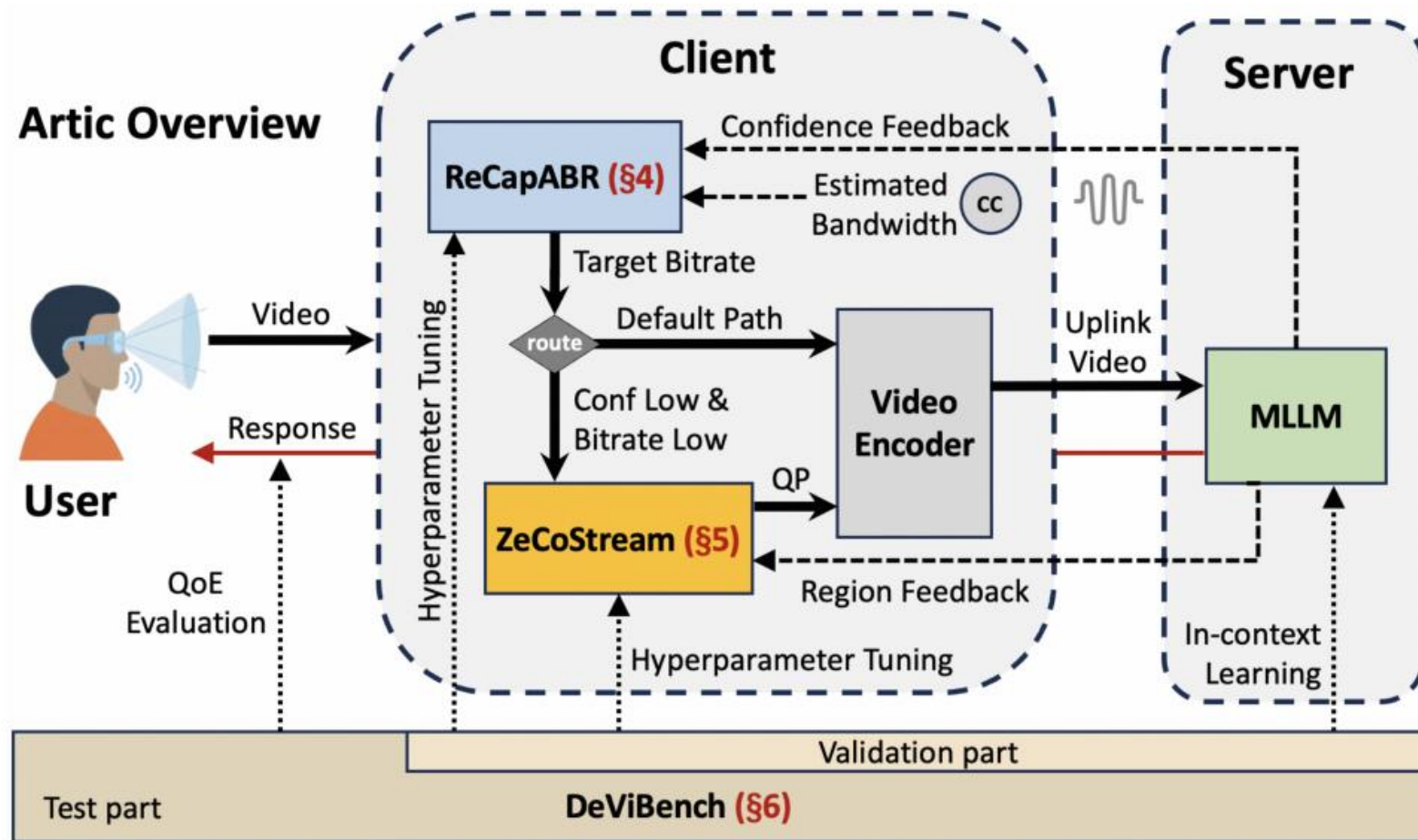
生成一些问答，再让大模型筛出高清对、低清错的题。通过另一个模型的交叉验证来确保正确性。

1968 道 QA

人工抽样100QA检验：93%正确

设计：Artic 的系统总览

端侧持续上传视频，云端 MLLM 返回两类反馈：回答置信度和重要区域。



两类反馈分别决定总码率和帧内质量；DeViBench检验答题准确率。

实验设置

以 WebRTC（传统的实时通信系统）为baseline，在真实 5G 上行轨迹重放下比较系统。

底层控制 分别使用 GCC / BBR（两种常用的拥塞控制算法）

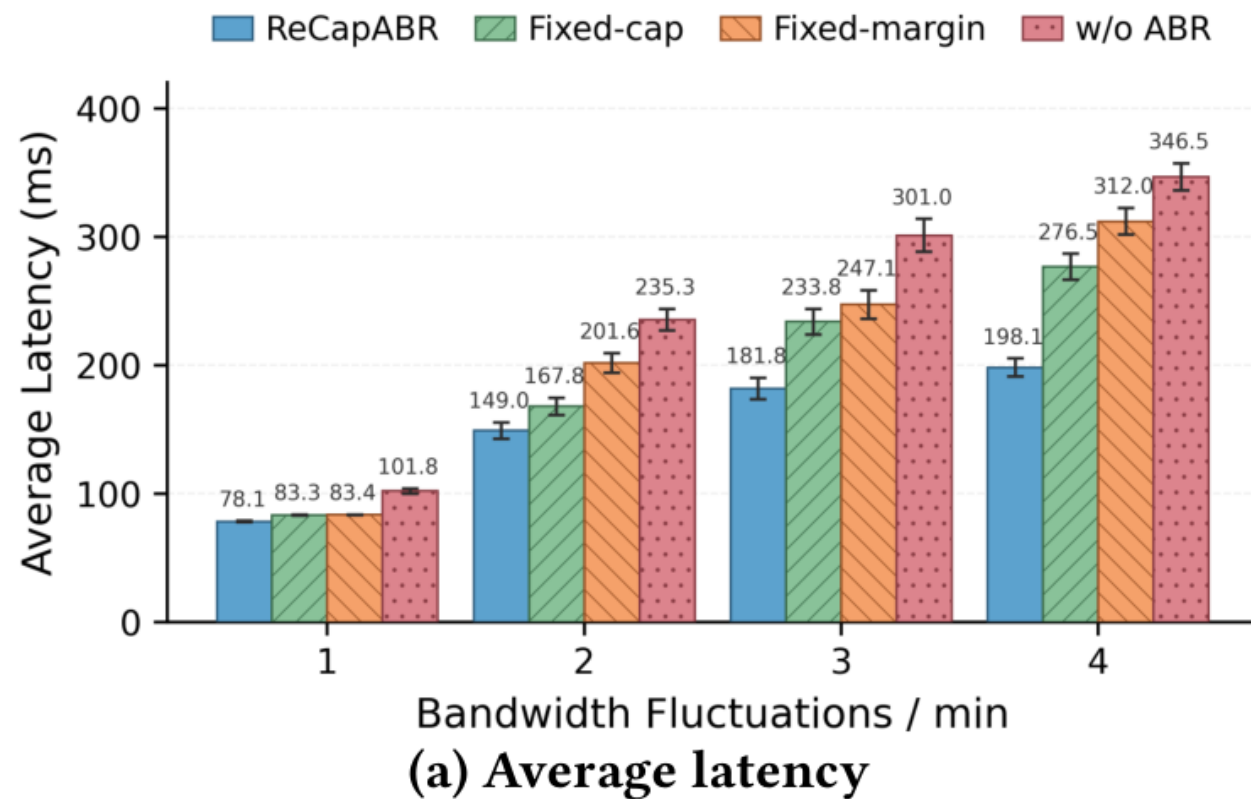
视频编码 H.265（一种广泛应用的视频编解码方式）

模型 GLM-4.6V-Flash 9B

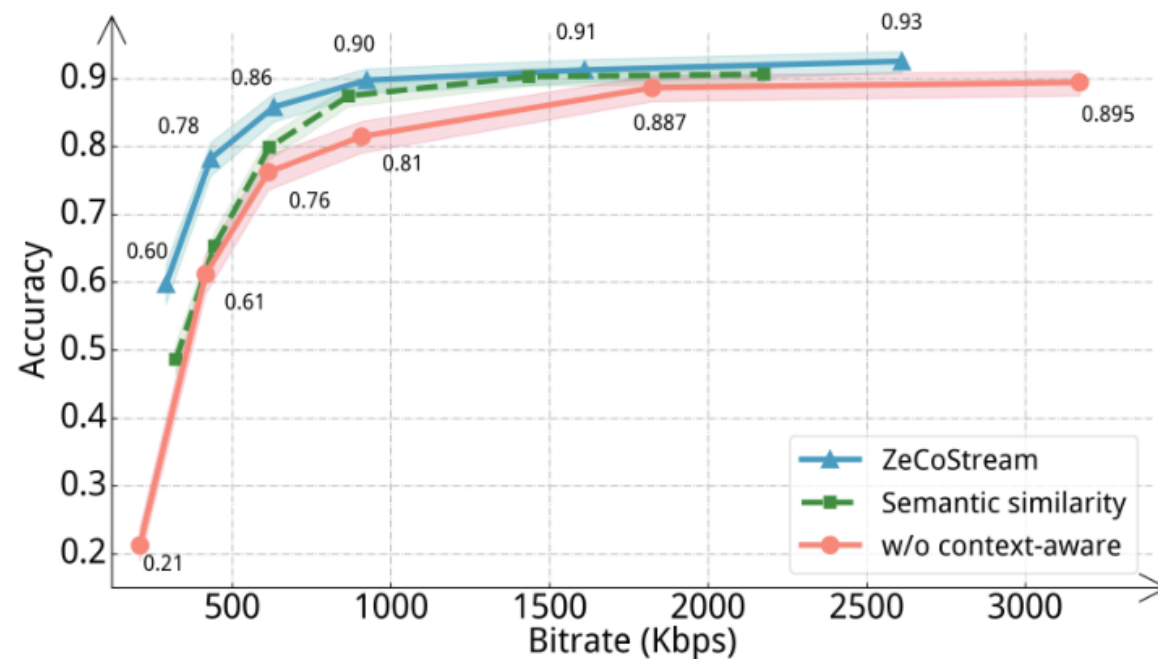
准确率 在 DeViBench 的问答题上统计答对比例

帧时延：客户端编码器 → 云端解码器；不含模型回答与语音回传。

实验结果：两个模块各自效果

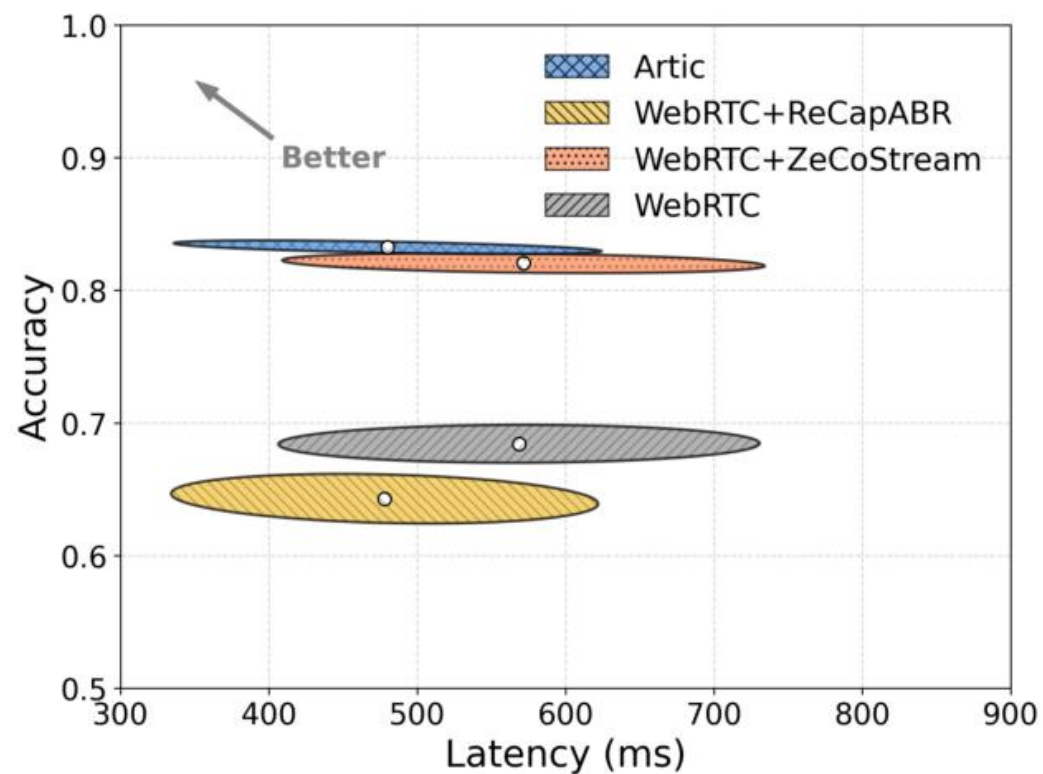
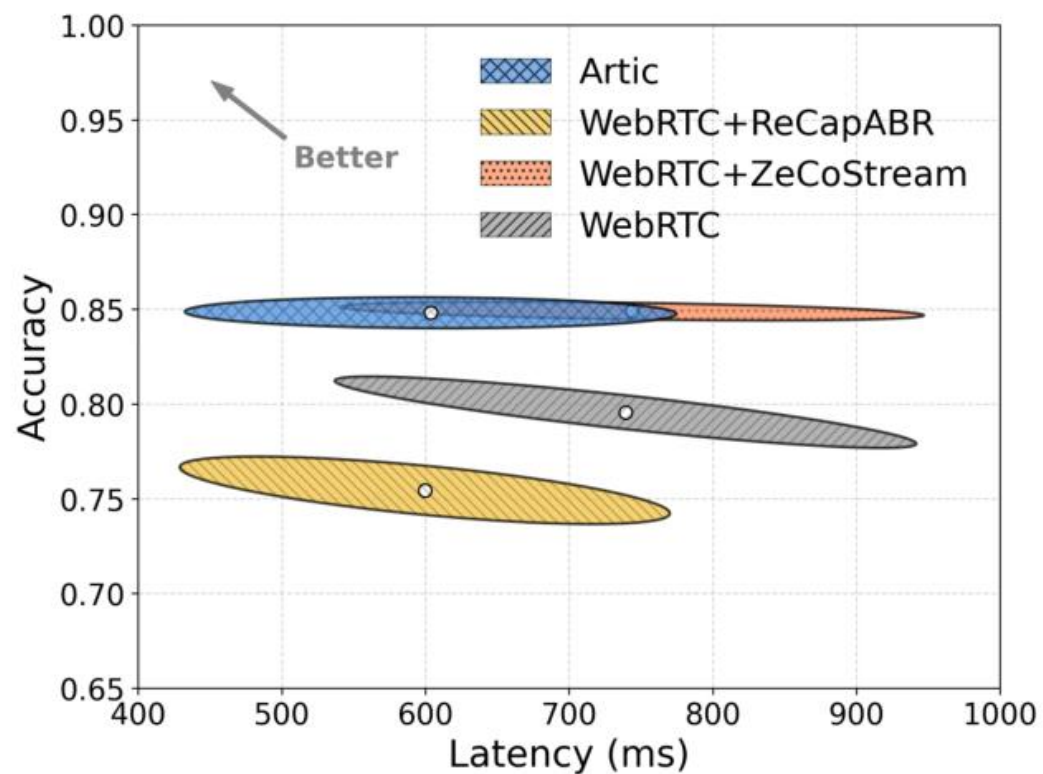


**ReCapABR: 波动越频繁,
节省的帧时延越多**



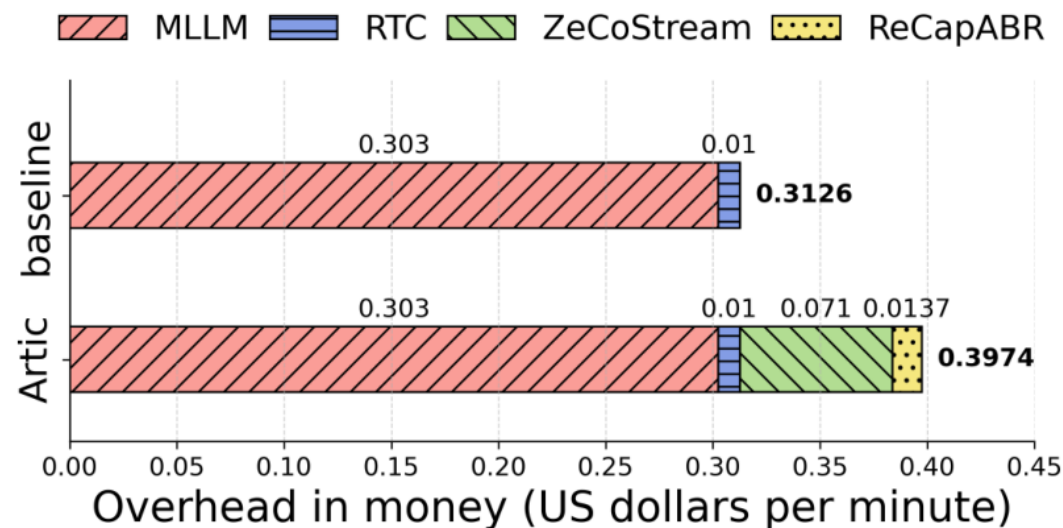
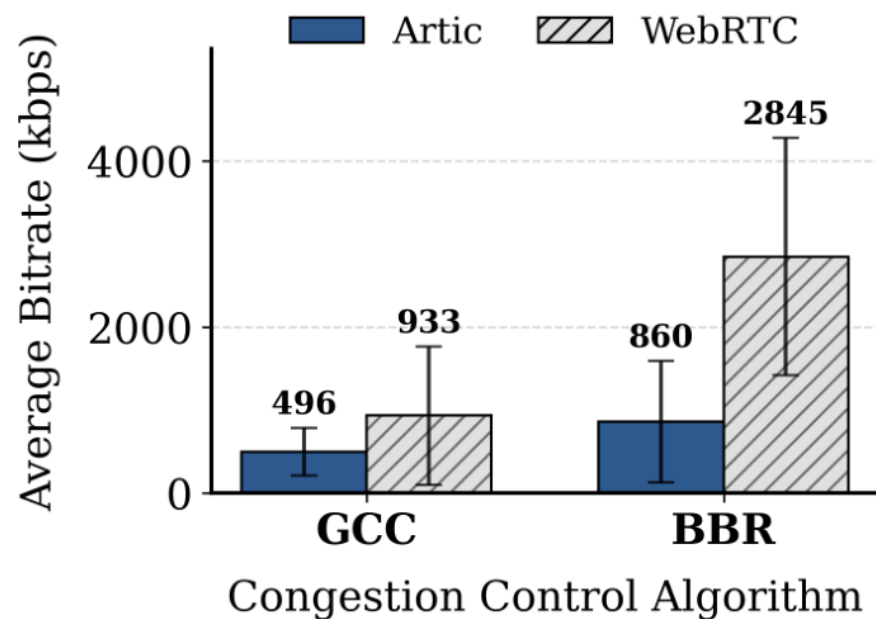
**ZeCoStream: 290 Kbps 时,
准确率 0.39 → 0.60**

实验结果：完整系统的准确率与帧时延



GCC 准确率 +15.12 个百分点; BBR 帧时延 -135.31 ms。

实验结果：上行流量和模型调用费用



GCC -46.84% | BBR -69.77%

\$0.3126 → \$0.3974 / min

端侧免额外识别模型；生成云端反馈的 API 费用约增 27.13%。

思考：提高与泛化

能不能进一步提高？

端侧与边缘节点有余力时，可尝试联合计算做轻量预处理：先筛画面变化和候选区域，再结合云端反馈，提前安排后续帧的局部质量。

问题可以泛化吗？

一个模型的输出交给另一个模型作为输入实际上已经广泛存在：agent workflow
为了提高模型准确率而做系统适配也已经广泛存在：harness相关工作
两个代表性的工作：RAG workflow 优化 / KV cache 传输

思考：能否用在我们的场景上

别人的好方法能不能用到我们的场景？

Artic

模型按回答需要定位细节，
把编码比特给关键区域



生成式视频传输

重建的小字、轮廓也容易损伤；
能否按任务决定额外传哪些残差？



Q&A

汇报人：陆俊安
2026.9.28